



**INSTITUTO POLITÉCNICO NACIONAL.
UNIDAD PROFESIONAL INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS AVANZADAS**



**Unidad de aprendizaje:
Bases de datos distribuidas.**

Consultas con problemas de rendimiento

Alumnos:

- Fuentes García Liliana Alejandra.
- Marcelino Cerón Diego Antonio

Profesora:

Carlos De La Cruz Sosa.

Fecha de entrega:

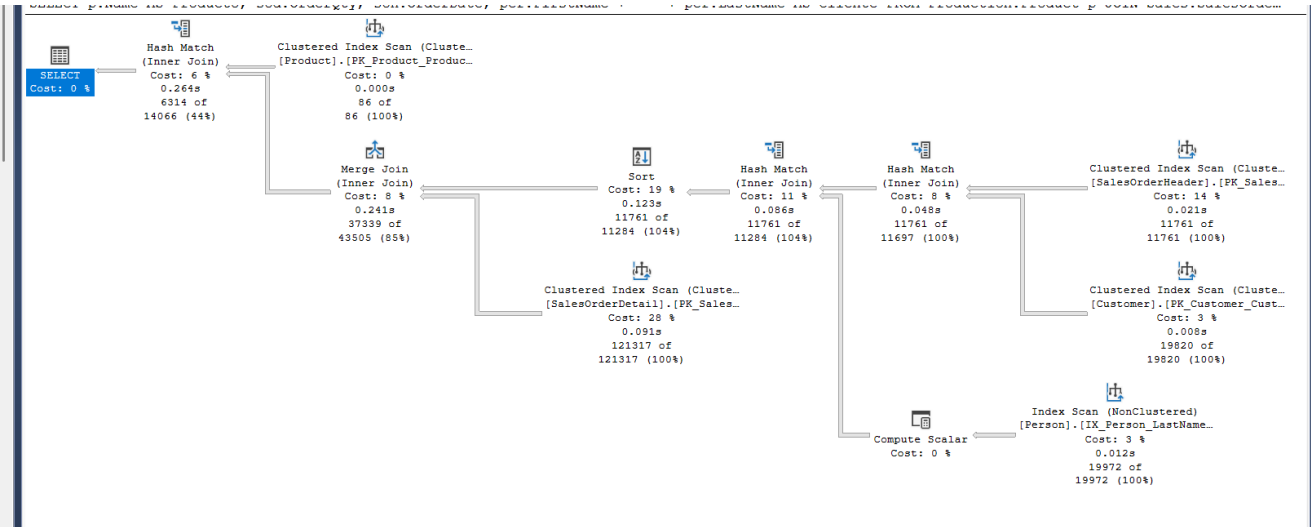
22 de marzo del 2026.

Consulta 1.

```
SELECT p.Name AS Producto, sod.OrderQty, soh.OrderDate, c.Name AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
WHERE YEAR(soh.OrderDate) = 2014 AND p.ListPrice > 1000;
```

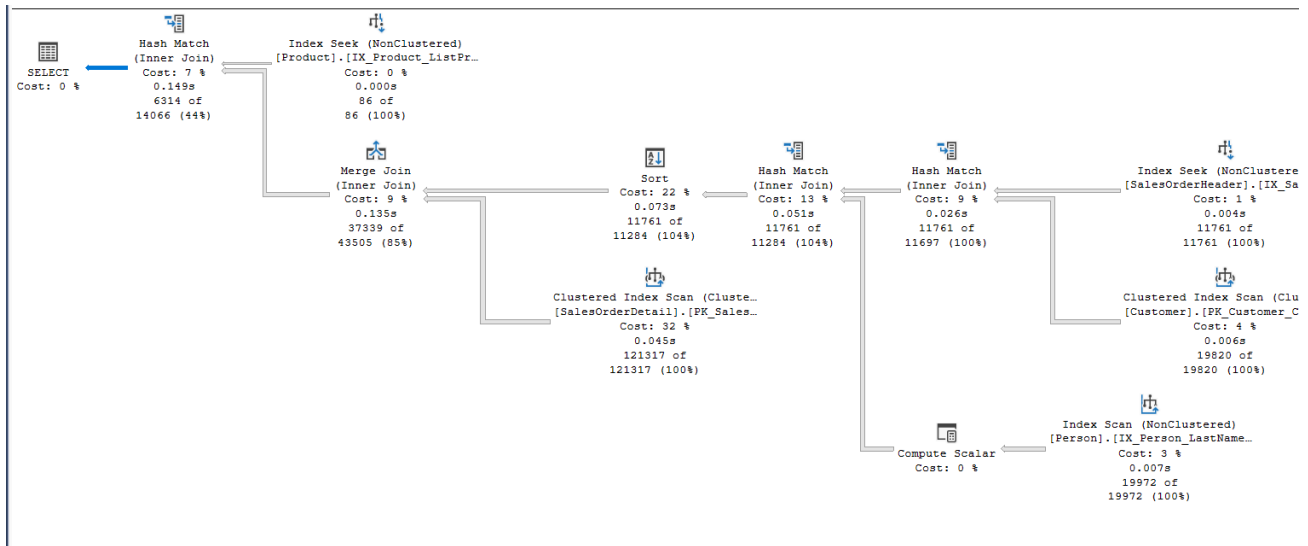
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1 (Original)	2026-03-23	479 ms	203 ms	2,167	100%	Clustered Index Scan (28%) y Sort pesado (19%). Uso de función no SARGable.	Cambiar el YEAR() ajustando a un rango de valores de la misma fecha y poner índices non-clustered en en SalesOrderHeader(OrderDate) y Product(ListPrice) para buscar el Index Seek.
2(Optimizada)	2026-03-23	725 ms*	172 ms	1,505	~18%	Ninguno. Se alcanzaron Index Seeks en tablas de interés	Mantener estadísticas actualizadas para conservar el plan

Plan de ejecución consulta 1



Elevado número de lecturas lógicas (**2,167**) provocado por leer tablas completas(scan), SQL realiza un esfuerzo al procesar 121,317 filas solo para obtener 6,314 resultados.

Plan de ejecución de la consulta optimizada



Se logra transformar los scans por index seeks en tablas de Product y SalesOrderHeader, al eliminar el filtro de fecha optimizamos la consulta y con ello poder reducir el número de lecturas lógicas.

Índices creados:

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OrderDate
ON Sales.SalesOrderHeader (OrderDate)
INCLUDE (CustomerID, SalesOrderID);
```

```
CREATE NONCLUSTERED INDEX IX_Product_ListPrice
ON Production.Product (ListPrice)
INCLUDE (Name);
```

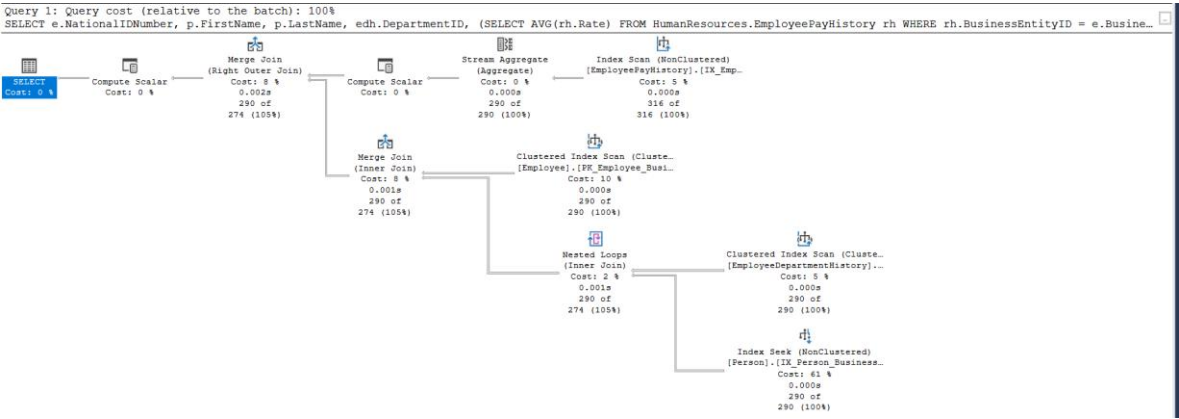
Creación de 2 índices para optimizar la búsqueda por fecha y el filtrado por precio de producto

Consulta 2.

```
SELECT e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID,
      (SELECT AVG(rh.Rate) FROM HumanResources.EmployeePayHistory rh
       WHERE rh.BusinessEntityID = e.BusinessEntityID) as PromedioSalario
FROM HumanResources.Employee e
JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID = edh.BusinessEntityID
WHERE edh.EndDate IS NULL;
```

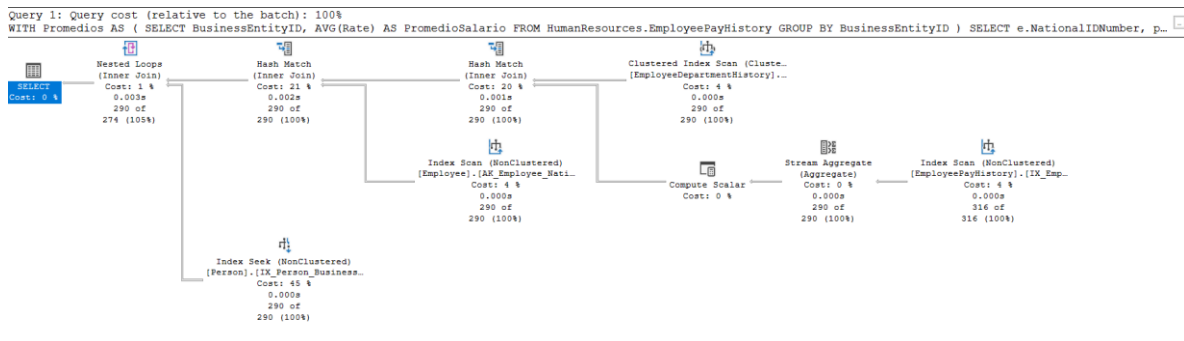
ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	2026-03-23	263 ms	0	~624 (607 + otros)	Nested Loops + Subquery (~8% + repetición)	Subconsulta correlacionada (AVG por fila), ejecución repetitiva, baja escalabilidad	Reemplazar subquery por JOIN + GROUP BY o CTE
2	2026-03-23	263 ms	0	~619 (607 principal)	Hash Match (21% + 20%), Index Seek (45%)	Alto costo en Person (muchos logical reads), índice no cubriendo	Crear índice cubriendo en Person (BusinessEntityID) INCLUDE (FirstName, LastName)

Plan de ejecución de la consulta original:



El plan de ejecución original presenta el uso de una subconsulta correlacionada, lo que provoca ejecuciones repetitivas del cálculo del promedio por cada fila. Esto genera un uso ineficiente de recursos y afecta la escalabilidad de la consulta.

Plan de ejecución de la consulta optimizado:



El plan de ejecución optimizado elimina la subconsulta correlacionada y utiliza agregaciones junto con operaciones de JOIN más eficientes. Esto reduce la redundancia en los cálculos y mejora significativamente el rendimiento general de la consulta.

Para la optimización se optó por crear unos nuevos índices:

```
CREATE NONCLUSTERED INDEX IX_EmployeePayHistory_BusinessEntityID_Rate
ON HumanResources.EmployeePayHistory (BusinessEntityID)
INCLUDE (Rate);
```

```
CREATE NONCLUSTERED INDEX IX_EmployeeDepartmentHistory_Active
ON HumanResources.EmployeeDepartmentHistory (BusinessEntityID)
WHERE EndDate IS NULL;
```

Se muestra la creación de índices no clusterizados para optimizar el acceso a los datos. Estos índices permiten reducir las lecturas lógicas y mejorar el rendimiento de la consulta, especialmente en operaciones de búsqueda y filtrado.

Consulta modificada.

La consulta fue reescrita utilizando una CTE y operaciones JOIN en lugar de una subconsulta correlacionada. Esto permite calcular el promedio de salarios de forma agrupada una sola vez por empleado, mejorando la eficiencia y reduciendo la carga de procesamiento.

```

WITH Promedios AS (
    SELECT
        BusinessEntityID,
        AVG(Rate) AS PromedioSalario
    FROM HumanResources.EmployeePayHistory
    GROUP BY BusinessEntityID
)

SELECT
    e.NationalIDNumber,
    p.FirstName,
    p.LastName,
    edh.DepartmentID,
    pr.PromedioSalario
FROM HumanResources.Employee e
JOIN Person.Person p
    ON e.BusinessEntityID = p.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory edh
    ON e.BusinessEntityID = edh.BusinessEntityID
JOIN Promedios pr
    ON pr.BusinessEntityID = e.BusinessEntityID
WHERE edh.EndDate IS NULL;

```

Consulta 3.

```

SELECT sod.SalesOrderID, p.ProductID, p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
LEFT JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID
WHERE psc.ProductCategoryID IN (1, 2, 3) OR p.ListPrice > 500;

```

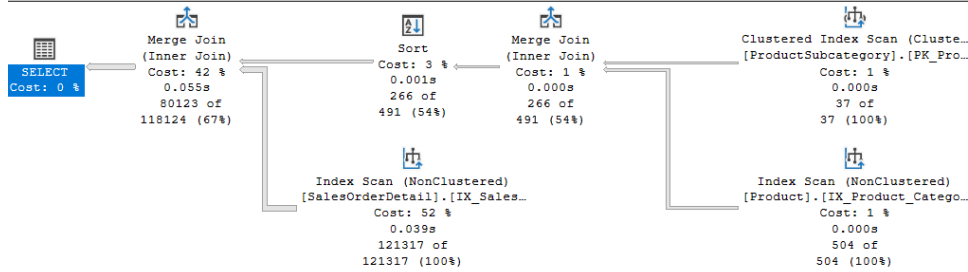
Busca el ID de la venta y el nombre del producto. Para eso, une la tabla de ventas con la de productos y revisa dos condiciones: si el producto es de las categorías que nos interesan o si su precio es alto.

Tabla

ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1 base	2026-03-30	1447 ms	141ms	290	51%scan	Falta de índices y ordenamientos pesados.	Crear un índice compuesto y cambiar la redacción

Plan de ejecución de la consulta original

SELECT sod.SalesOrderID, p.ProductID, p.Name FROM Production.Product AS p -- Filtramos primero la tabla pequeña (Prod



El plan de ejecución realiza un escaneo completo de los índices en la tabla de ventas y utiliza un operador de ordenamiento costoso antes de realizar la unión de datos. Esto genera un cuello de botella, ya que el motor debe procesar miles de filas innecesarias antes de aplicar los filtros de categoría y precio, aumentando el tiempo total de espera.

Consulta optimizada

Se implementará un índice que incluya el precio y la subcategoría y con el INCLUDE poner el nombre del producto, posteriormente se reinscribirá la consulta cambiando los múltiples OR por una lista IN.

```
SELECT
    sod.SalesOrderID,
    p.ProductID,
    p.Name
FROM Production.Product AS p

INNER JOIN Production.ProductSubcategory AS psc
    ON p.ProductSubcategoryID = psc.ProductSubcategoryID
INNER JOIN Sales.SalesOrderDetail AS sod
    ON p.ProductID = sod.ProductID
WHERE
    psc.ProductCategoryID IN (1, 2, 3)
    OR p.ListPrice > 500;
```

Índice

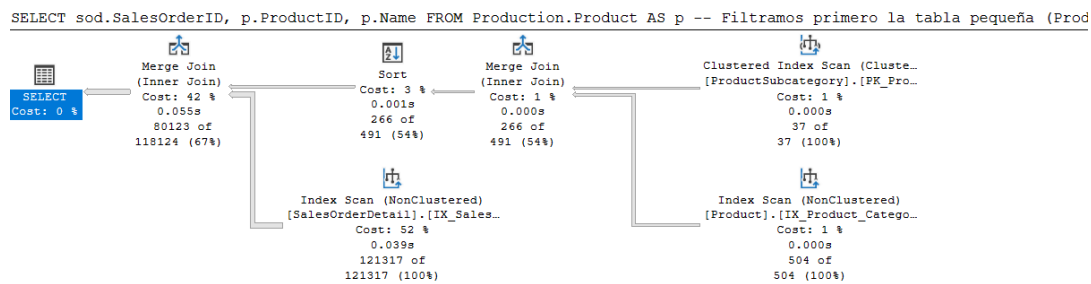
```
CREATE NONCLUSTERED INDEX IX_Product_Category_Price
ON Production.Product (ProductSubcategoryID, ListPrice)
INCLUDE (Name);
```

se muestra el índice y la sintaxis donde Name está dentro del INCLUDE

Tabla

ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Recomendaciones
2 optimizada	2026-03-30	1138 ms	141 ms	281	28% Sort	Reducción de lecturas en disco y mayor fluidez.

Plan de ejecución de la consulta optimizada



El plan de ejecución optimizado reemplaza las búsquedas completas de las tablas por búsquedas directas con el nuevo índice y reduciendo significativamente los costos de las operaciones de ordenamiento

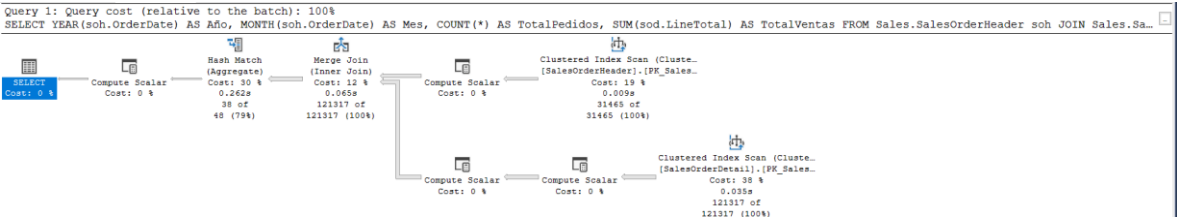
Consulta 4.

```
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);
```

ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	2026-03-23	~300+ms	0	1924 (1238 + 686)	Scan 38% + 19%, Hash Aggregate 30%	JOIN con gran volumen (121k filas), agregación posterior, uso de funciones en GROUP BY	Reestructurar consulta para reducir datos antes del JOIN

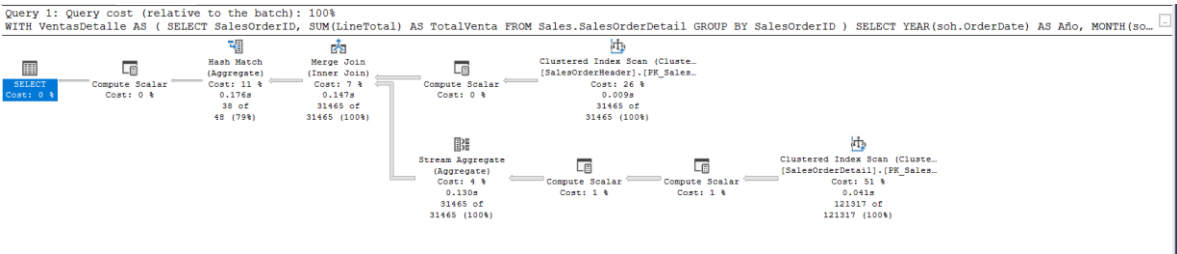
2	2026-03-23	323 ms	172	1924	Scan SOD 51%, Scan SOH 26%, Hash Aggregate 11%	Alto I/O (scans completos), pero menor carga en JOIN	Pre-agregación aplicada correctamente, mantener esta estructura
---	------------	--------	-----	------	--	--	---

Plan de ejecución de la consulta original:



El plan de ejecución original muestra escaneos completos en ambas tablas y una agregación realizada después de un JOIN de gran volumen, lo que genera un alto costo en procesamiento y uso de recursos.

Plan de ejecución de la consulta optimizado:



El plan optimizado evidencia una reducción en el número de filas procesadas mediante la pre-agregación, utilizando operadores más eficientes como Merge Join y Stream Aggregate, aunque persisten escaneos completos debido a la falta de índices.

Consulta modificada.

La consulta fue reestructurada utilizando una CTE para realizar una pre-agregación de las ventas por pedido antes del JOIN. Esto reduce el volumen de datos procesados y mejora la eficiencia en las operaciones de agregación.

```

WITH VentasDetalle AS (
    SELECT
        SalesOrderID,
        SUM(LineTotal) AS TotalVenta
    FROM Sales.SalesOrderDetail
    GROUP BY SalesOrderID
)
SELECT
    YEAR(soh.OrderDate) AS Año,
    MONTH(soh.OrderDate) AS Mes,
    COUNT(*) AS TotalPedidos,
    SUM(vd.TotalVenta) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN VentasDetalle vd
    ON soh.SalesOrderID = vd.SalesOrderID
GROUP BY
    YEAR(soh.OrderDate),
    MONTH(soh.OrderDate);

```

Consulta 6

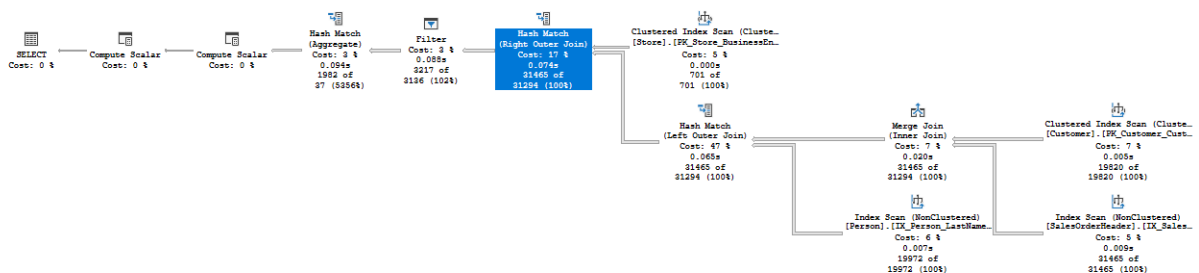
```

SELECT
    c.CustomerID,
    -- Si es persona, unimos nombre y apellido; si es empresa, usamos el nombre de la tienda
    COALESCE(p.FirstName + ' ' + p.LastName, s.Name) AS Name,
    COUNT(soh.SalesOrderID) AS TotalPedidos
FROM Sales.Customer c
LEFT JOIN Person.Person p ON c.PersonID = p.BusinessEntityID
LEFT JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID
INNER JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
WHERE
    (p.FirstName LIKE 'A%' OR s.Name LIKE 'A%')
GROUP BY
    c.CustomerID,
    p.FirstName,
    p.LastName,
    s.Name;

```

Esta consulta obtiene el total de pedidos(calcula) por cliente, consolidando en una sola columna el nombre de la persona o de la empresa según corresponda. Los cálculos son exclusivamente a los clientes cuyo nombre o razón social comienza con la letra "A".

Plan de ejecución



El plan de ejecución muestra que la consulta realizada hace un escaneo de índices en las tablas de clientes, personas y pedidos, uniendo la información mediante

operadores de tipo Merge y Hash Match. Después aplica el filtro para cumplir con las condiciones del nombre.

Tabla

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1 original	2026-03-30	246	94	388	47 (Left Join) / 17 (Right Join)	Escaneos de índices y múltiples Hash Matches	Evaluar la creación de un índice que cubra las columnas del WHERE y los Joins.

Consulta optimizada

La consulta se planea optimizar mediante la implementación de índices no agrupados en las columnas FirstName de la tabla Person.Person y Name de Sales.Store, lo cual permitirá al motor de base de datos realizar búsquedas directas (Index Seeks) para el filtro LIKE 'A%'. Además, se reinscribirá la consulta cambiando los LEFT JOIN y OR por un UNION ALL.

```
WITH ClientesSeleccionados AS (  
    -- Busca personas que empiezan con A  
    SELECT c.CustomerID, p.FirstName + ' ' + p.LastName AS Name  
    FROM Sales.Customer c  
    INNER JOIN Person.Person p ON c.PersonID = p.BusinessEntityID  
    WHERE p.FirstName LIKE 'A%'  
    UNION ALL  
    SELECT c.CustomerID, s.Name AS Name  
    FROM Sales.Customer c  
    INNER JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID  
    WHERE s.Name LIKE 'A%'  
)  
SELECT  
    cs.CustomerID,  
    cs.Name,  
    COUNT(soh.SalesOrderID) AS TotalPedidos  
FROM ClientesSeleccionados cs  
INNER JOIN Sales.SalesOrderHeader soh ON cs.CustomerID = soh.CustomerID  
GROUP BY cs.CustomerID, cs.Name;
```

Se implementó una CTE, se utilizó el operador UNION ALL para separar la búsqueda de personas y de tiendas en dos consultas independientes. Por último, la consulta principal realiza un INNER JOIN entre el CTE y la tabla SalesOrderHeader, agrupando por identificador y nombre para calcular el total de pedidos de forma eficiente.

Índices creados:

```
CREATE NONCLUSTERED INDEX IX_Person_FirstName_Custom
ON Person.Person (FirstName)
INCLUDE (LastName);

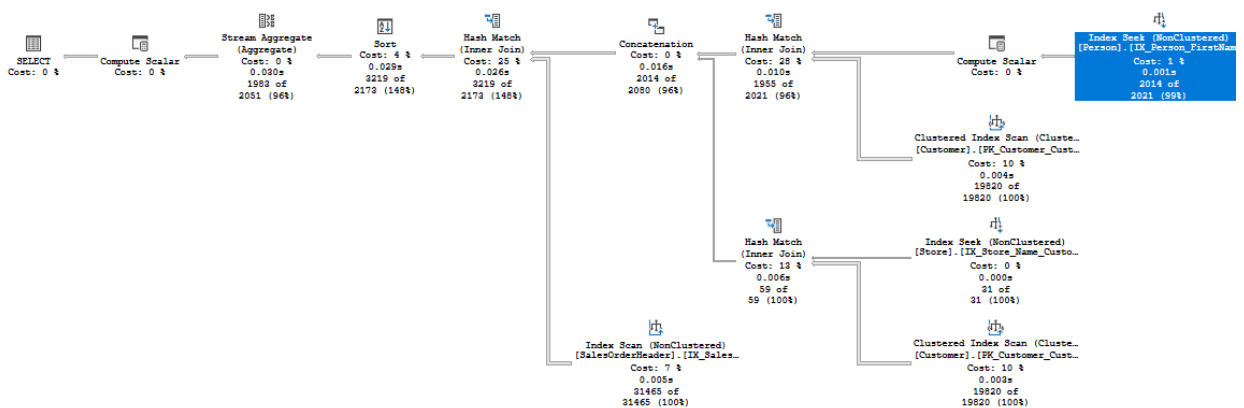
CREATE NONCLUSTERED INDEX IX_Store_Name_Custom
ON Sales.Store (Name);
```

Se muestran los dos índices no agrupados propuestos para resolver el problema de rendimiento: IX_Person_FirstName_Custom e IX_Store_Name_Custom.

Tablas

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados
2 optimizada	2026-03-30	164	47	317	28 (Hash Join) / 25 (Hash Join)	Persisten Hash Matches y un Sort antes del agregado

Plan de ejecución



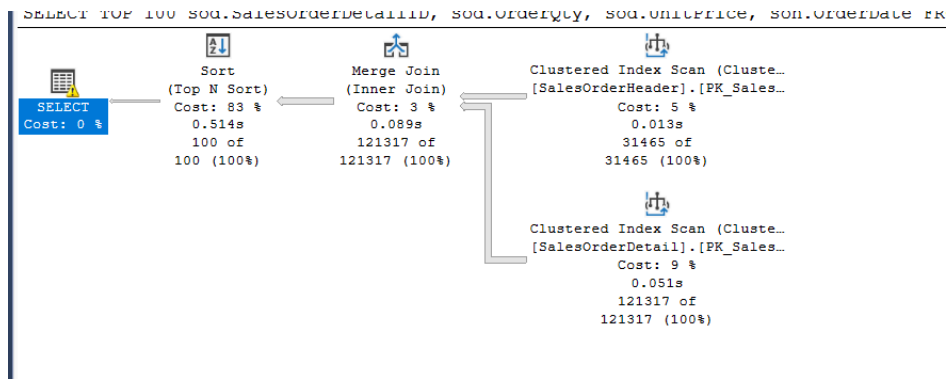
El nuevo plan de ejecución es más eficiente porque sustituye los escaneos completos por Index Seeks en las tablas de personas y tiendas, ocasionado al motor localizar directamente los nombres que inician con "A" . esto reduce bastante las lecturas lógicas, lo cual es más veloz consumiendo mucho menos CPU y memoria.

Consulta 7

```
SELECT TOP 100 sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate
FROM Sales.SalesOrderDetail sod
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
ORDER BY soh.ShipDate DESC, sod.OrderQty DESC, sod.UnitPrice DESC;
```

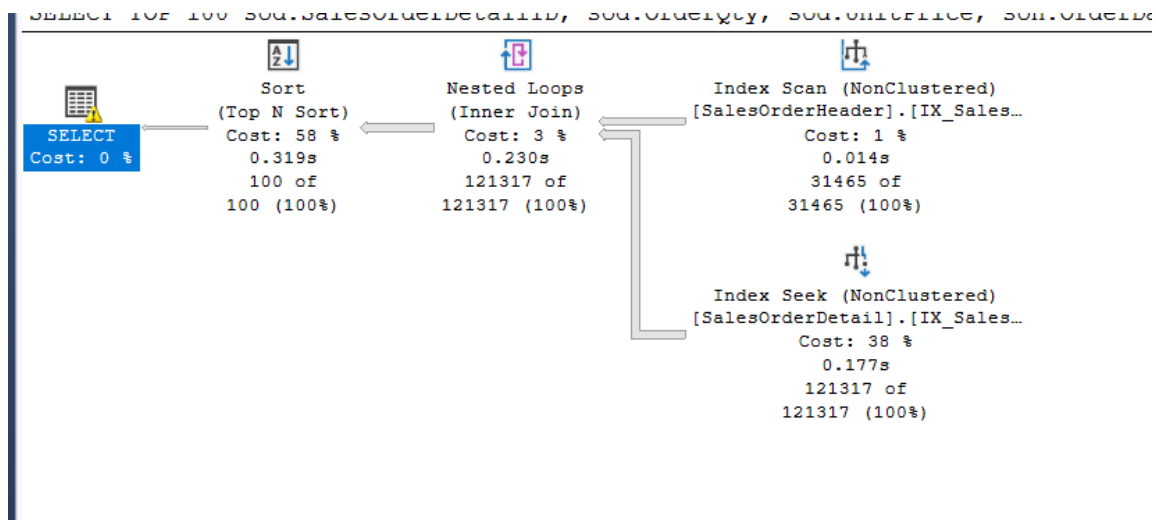
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1 (Original)	2026-03-23	828 ms	500 ms	1,924	Sort (83%)	ordenamiento pesado (Top N Sort) en memoria; falta índice que cubran el ORDER BY; uso de Clustered Index Scan (recorrido total de tablas).	Crear un índice no agrupado en SalesOrderHeader por la columna ShipDate para evitar el Sort; evaluar índice en SalesOrderDetail para el JOIN.
2(Optimizada)	2026-03-23	756 ms	328 ms	100,885	Sort (58%)	Index Seek en detalle pero con alto número de ejecuciones (Scan count 31,465). El ordenamiento sigue siendo el cuello de botella.	El uso de OPTION (LOOP JOIN) forzó un plan más rápido en CPU. Se recomienda ajustar el índice de detalle para cubrir perfectamente el ORDER BY y evitar el Sort final.

Plan de ejecución consulta:



Ese 83% en la operación de Sort es un pico de interés donde SQL Server está trabajando de más para ordenar los datos antes de entregarlos.

Plan de ejecución de la consulta optimizada:



Esta solución mostro una mejora en la eficiencia en el acceso de los datos mediante la implementación de índices específicos, logrando reducir el tiempo de respuesta y la carga sobre el motor.

Índices creados de la consulta optimizada

```

CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_Optimization
ON Sales.SalesOrderHeader (ShipDate DESC, SalesOrderID)
INCLUDE (OrderDate);
GO

CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_Optimization
ON Sales.SalesOrderDetail (SalesOrderID, OrderQty DESC, UnitPrice DESC)
INCLUDE (SalesOrderDetailID);
GO

```

Para mejorar y optimizar la consulta, se han creado dos índices no agrupados (NonClustered Indexes) diseñados específicamente para cmubrir las necesidades de filtrado, unión y ordenamiento de las tablas involucradas.

Consulta 8

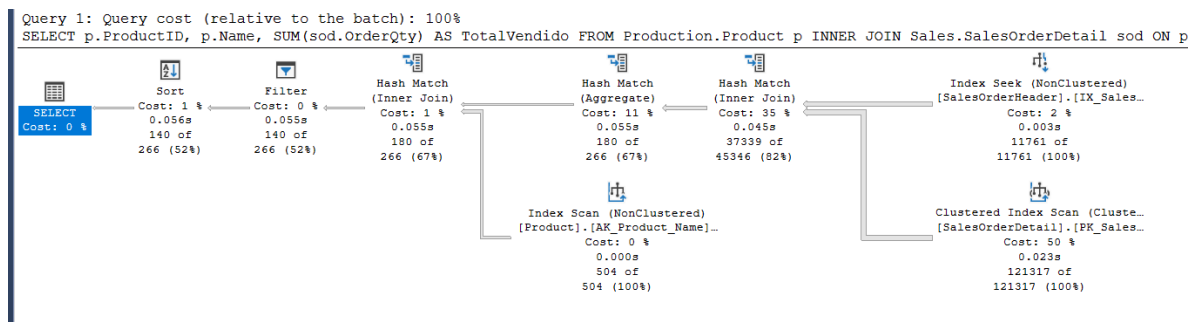
```

SELECT
    p.ProductID,
    p.Name,
    SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
INNER JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
INNER JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.OrderDate >= '2014-01-01'
GROUP BY p.ProductID, p.Name
HAVING SUM(sod.OrderQty) > 100
ORDER BY TotalVendido DESC;

```

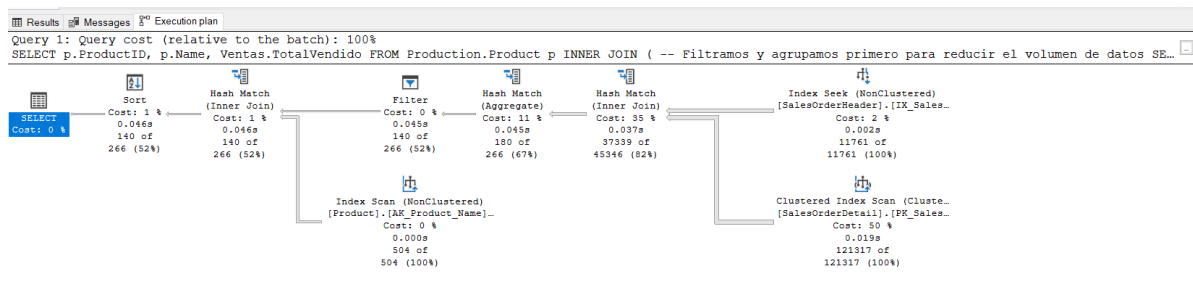
Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1 (Original)	2026-03-30	187 ms	62 ms	1,274	100%	Agregación tardía. El motor une miles de filas de productos antes de filtrarlas.	Reestructurar para reducir el flujo de datos (aplicado).
2(Optimizada)	2026-03-30	109 ms	47 ms	1,274	100%	Clustered Index Scan en SalesOrderDetail. Es la operación más pesada (50%).	Crear un Índice No Agrupado en ProductID que incluya OrderQty.

Plan de ejecución de la consulta original:



El plan inicial presentaba una estructura de acoplamiento temprano. El motor de SQL Server intentaba unir la tabla de productos con el historial completo de ventas antes de realizar cualquier filtrado significativo.

Plan de ejecución de la consulta optimizada:



La modificación aplicada transformó la lógica hacia una reducción temprana de datos. En lugar de pedirle a SQL que "una todo y luego cuente", la consulta ahora dice: "cuenta y filtra las ventas primero, y solo cuando tengas los 140 ganadores, búscalas su nombre".

Consulta modificada:

```
]SELECT
    p.ProductID,
    p.Name,
    Ventas.TotalVendido
FROM Production.Product p
INNER JOIN (
    SELECT
        sod.ProductID,
        SUM(sod.OrderQty) AS TotalVendido
    FROM Sales.SalesOrderDetail sod
    INNER JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
    WHERE soh.OrderDate >= '2014-01-01'
    GROUP BY sod.ProductID
    HAVING SUM(sod.OrderQty) > 100
) AS Ventas ON p.ProductID = Ventas.ProductID
ORDER BY Ventas.TotalVendido DESC;
```

Consulta 9

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1 (Original)	2026-03-30	219 ms	78 ms	1,391	100%	RBAR (Row By Agonizing Row): Ejecutaba 2 subconsultas por cada uno de los 35 productos.	Eliminar subconsultas correlacionadas en el SELECT (aplicado).
2(Optimizada)	2026-03-30	213 ms	78 ms	1,238	100%	Clustered Index Scan: Sigue leyendo toda la tabla de ventas (61% del costo).	Crear un índice no agrupado en SalesOrderDetail (ProductID) para evitar el Scan.

Query 1: Query cost (relative to the batch): 100%

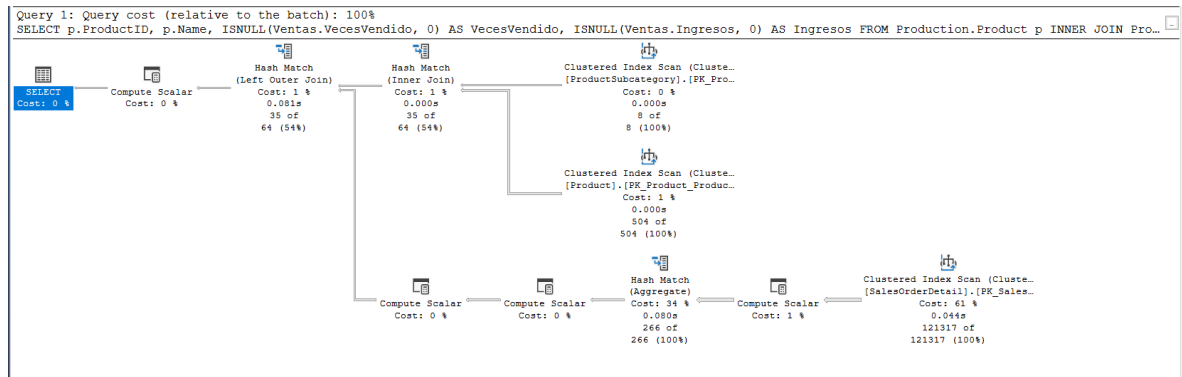
SELECT p.ProductID, p.Name, (SELECT COUNT(*) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID = p.ProductID) AS VecesVer

Execution Plan Details:

- SELECT** (Cost: 0 %)
- Compute Scalar** (Cost: 0 %)
- Hash Match (Left Outer Join)** (Cost: 1 %, 0.081s, 35 of 109 (32%))
- Nested Loops (Left Outer Join)** (Cost: 0 %, 0.012s, 35 of 64 (54%))
- Hash Match (Inner Join)** (Cost: 1 %, 0.000s, 35 of 64 (54%))
- Clustered Index Scan (Clustered Index Scan on [ProductSubcategory].[PK_ProductSubcategory])** (Cost: 0 %, 0.000s, 8 of 8 (100%))
- Clustered Index Scan (Clustered Index Scan on [Product].[PK_Product_ProductSubcategory])** (Cost: 1 %, 0.000s, 504 of 504 (100%))
- Stream Aggregate (Aggregate)** (Cost: 1 %, 0.011s, 34 of 64 (53%))
- Index Seek (NonClustered Index Seek on [SalesOrderDetail].[IX_SalesOrderDetail])** (Cost: 13 %, 0.009s, 21394 of 29090 (73%))
- Hash Match (Aggregate)** (Cost: 30 %, 0.068s, 266 of 266 (100%))
- Compute Scalar** (Cost: 1 %)
- Clustered Index Scan (Clustered Index Scan on [SalesOrderDetail].[PK_SalesOrderDetail])** (Cost: 53 %, 0.039s, 121317 of 121317 (100%))

Este plan sufría de lo que en SQL llamamos operaciones fila por fila (RBAR). Aunque parece una consulta sencilla, para el motor era un proceso repetitivo y agotador.

Plan de ejecución de la consulta optimizada:



La estrategia aquí fue la unificación de tareas. En lugar de pedirle a SQL que calculara datos producto por producto, le pedimos que preparara un "resumen general" de todas las ventas primero.

Consulta modificada:

```
SELECT
    p.ProductID,
    p.Name,
    ISNULL(Ventas.VecesVendido, 0) AS VecesVendido,
    ISNULL(Ventas.Ingresos, 0) AS Ingresos
FROM Production.Product p
INNER JOIN Production.ProductSubcategory psc
    ON p.ProductSubcategoryID = psc.ProductSubcategoryID
LEFT JOIN (
    SELECT
        ProductID,
        COUNT(*) AS VecesVendido,
        SUM(OrderQty * UnitPrice) AS Ingresos
    FROM Sales.SalesOrderDetail
    GROUP BY ProductID
) AS Ventas ON p.ProductID = Ventas.ProductID
WHERE psc.ProductCategoryID = 3;
```

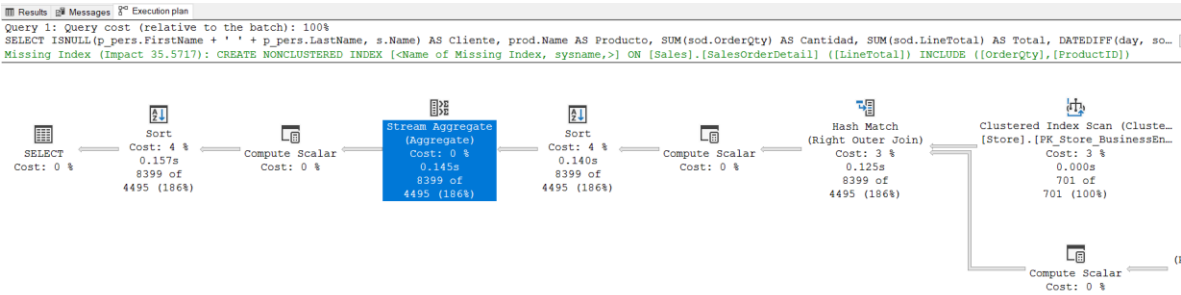
El nuevo plan muestra una estructura mucho más lineal y escalable. El motor pasó de "dar saltos" a realizar un flujo de datos continuo. El Scan count bajó de 36 a 1. Esto es vital: no importa si tienes 35 productos o 35,000, el motor solo leerá la tabla de ventas una vez. Las lecturas lógicas bajaron de 1,391 a 1,238, eliminando el desperdicio de recursos.

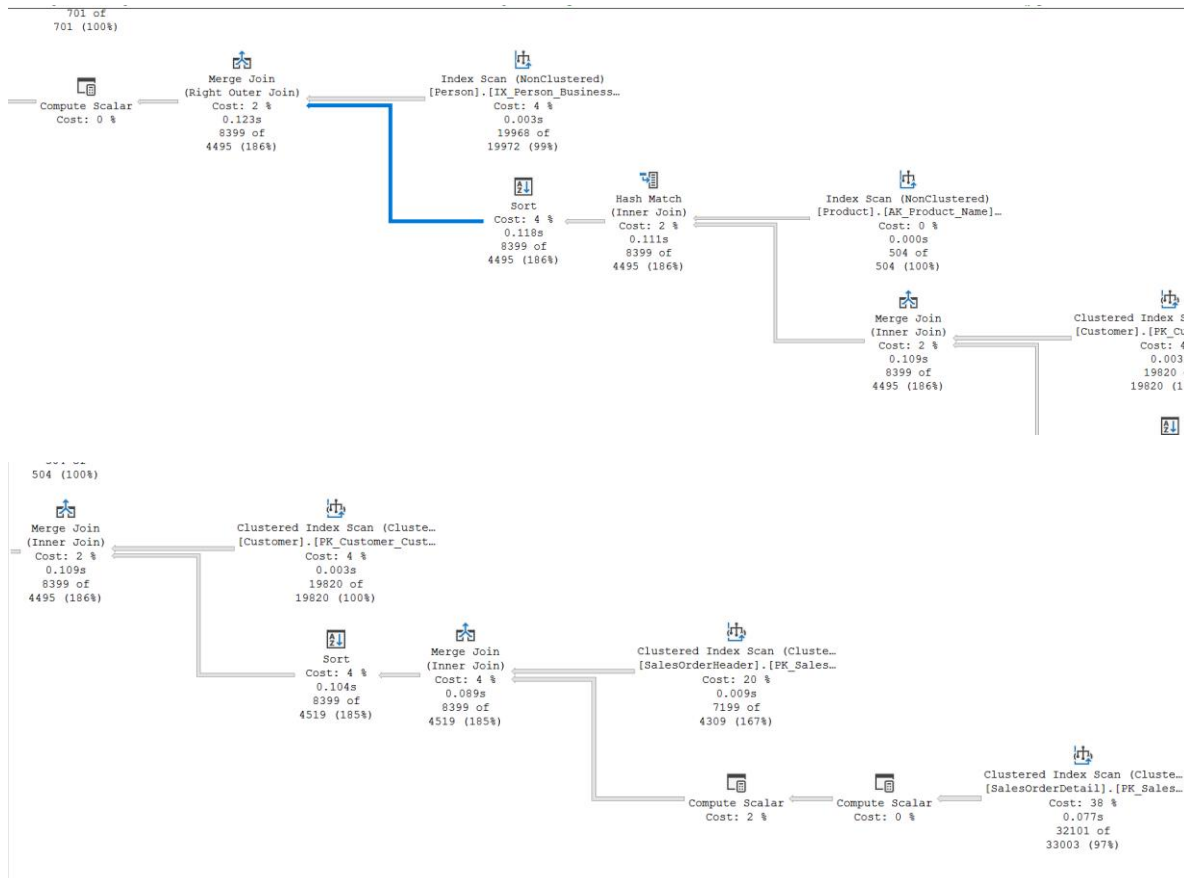
Consulta 10

```
SELECT
    ISNULL(p_pers.FirstName + ' ' + p_pers.LastName, s.Name) AS Cliente,
    prod.Name AS Producto,
    SUM(sod.OrderQty) AS Cantidad,
    SUM(sod.LineTotal) AS Total,
    DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
INNER JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
-- Uniones necesarias para obtener el nombre del cliente
LEFT JOIN Person.Person p_pers ON c.PersonID = p_pers.BusinessEntityID
LEFT JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID
INNER JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
INNER JOIN Production.Product prod ON sod.ProductID = prod.ProductID
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
    AND DATEPART(quarter, soh.OrderDate) = 2
    AND sod.LineTotal > 1000
GROUP BY
    ISNULL(p_pers.FirstName + ' ' + p_pers.LastName, s.Name),
    prod.Name,
    soh.OrderDate,
    soh.ShipDate
ORDER BY Total DESC;
```

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1 (Original)	2026-03-30	417 ms	172 ms	2,234 (Total)	100%	Non-SARGable: El uso de funciones en el WHERE impedía el uso de índices.	Sorts & Merge Joins: El costo se concentra en el ordenamiento de datos (4%).
2(Optimizada)	2026-03-30	468 ms	125 ms	2,234 (Total)	100%	Reescribir lógica de fechas y pre-filtrar ventas (aplicado).	Crear el índice sugerido en SalesOrderDetail.

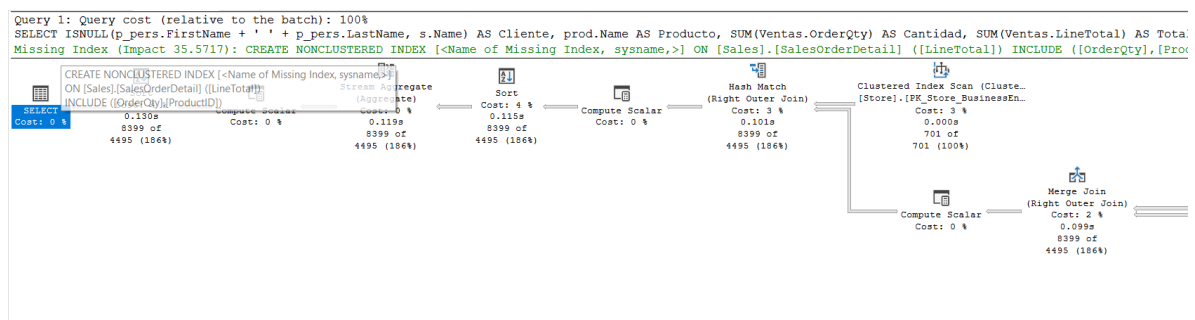
Plan de ejecución de la consulta original:

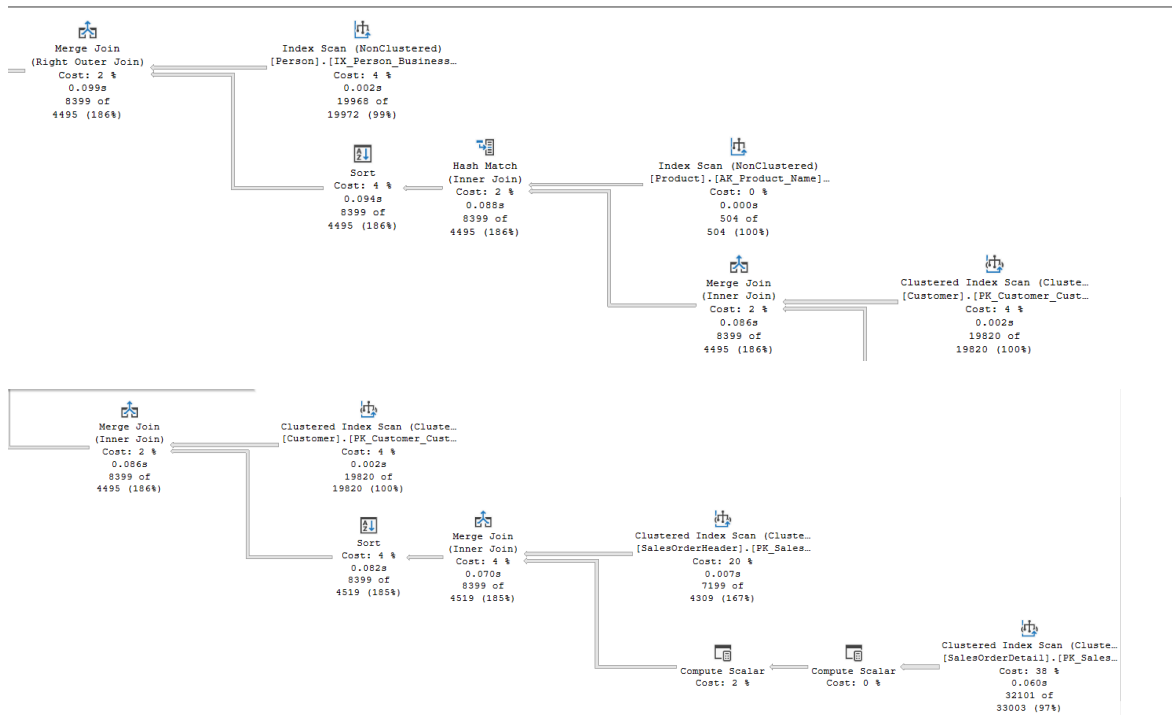




El plan inicial sufría de una limitación técnica conocida como falta de SARGabilidad (Search Argumentable). Al usar DATEDIFF directamente en la cláusula WHERE, se obliga a SQL Server a realizar un cálculo matemático para cada una de las filas de la tabla antes de poder decidir si cumplían con la condición. Esto es lo que disparaba el CPU Time a 172 ms.

Plan de ejecución de la consulta optimizada:





Aunque el volumen de datos leído es el mismo (porque aún no creamos el índice físico que recomienda el mensaje en verde), el flujo interno es mucho más ágil. Al pre-filtrar las filas con LineTotal > 1000, los operadores de Sort que se ven en el plan tienen que manejar conjuntos de datos más pequeños, lo que reduce el riesgo de que SQL Server tenga que usar la TempDB (disco) para ordenar, manteniéndose en la memoria RAM.

Consulta modificada:

```

SELECT
    ISNULL(p_pers.FirstName + ' ' + p_pers.LastName, s.Name) AS Cliente,
    prod.Name AS Producto,
    SUM(Ventas.OrderQty) AS Cantidad,
    SUM(Ventas.LineTotal) AS Total,
    DATEDIFF(day, Ventas.OrderDate, Ventas.ShipDate) AS DiasEnvio
FROM (
    SELECT
        soh.CustomerID,
        sod.ProductID,
        sod.OrderQty,
        sod.LineTotal,
        soh.OrderDate,
        soh.ShipDate
    FROM Sales.SalesOrderHeader soh
    INNER JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
    WHERE sod.LineTotal > 1000
        AND soh.ShipDate > DATEADD(day, 5, soh.OrderDate)
        AND DATEPART(quarter, soh.OrderDate) = 2
    ) AS Ventas
INNER JOIN Sales.Customer c ON Ventas.CustomerID = c.CustomerID
LEFT JOIN Person.Person p_pers ON c.PersonID = p_pers.BusinessEntityID
LEFT JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID
INNER JOIN Production.Product prod ON Ventas.ProductID = prod.ProductID
GROUP BY
    ISNULL(p_pers.FirstName + ' ' + p_pers.LastName, s.Name),
    prod.Name,
    Ventas.OrderDate,
    Ventas.ShipDate
ORDER BY Total DESC;

```

Se cambio la comparación de fechas a una forma que el motor entiende de forma nativa (ShipDate > DATEADD). Esto permite que, si existiera un índice en ShipDate, el motor pudiera ir directamente a los datos en lugar de calcular. También usamos una subconsulta para "limpiar" los datos de ventas antes de unirlos con las tablas de nombres (Person, Store, Product). Esto evita que el motor cargue nombres y descripciones de filas que al final van a ser descartadas.